

PROGRAMACIÓN II

TPI



Carrera: Tecnicatura Universitaria en Programación

Materia: Programación 2

Proyecto: Food Store - Sistema de Gestión de Pedidos de Comida

Alumno: Puca Kevin

Legajo: 53979

Año: 2026

ÍNDICE GENERAL

1. INTRODUCCIÓN Y OBJETIVOS

..... 1

- 1.1. Contextualización del Proyecto (Food Store) 1
- 1.2. Objetivos Generales y Académicos 1
- 1.3. Alcance y Límites del Sistema de Consola 2

2. ARQUITECTURA DEL SISTEMA Y ESTRUCTURA DE CAPAS 3

- 2.1. Justificación del Diseño Orientado a Objetos (POO) 3
- 2.2. Descripción de Paquetes y Responsabilidades 4
 - 2.2.1. Paquete config: Inicialización y Conexión 4
 - 2.2.2. Paquete entities: Modelo de Dominio y Herencia 4
 - 2.2.3. Paquete enums: Enumeraciones Estructuradas 5
 - 2.2.4. Paquete dao: Patrón de Acceso a Datos (JDBC) 5
 - 2.2.5. Paquete service: Capa de Lógica y Validaciones 6
 - 2.2.6. Paquete exception: Manejo de Errores Personalizados 6
 - 2.2.7. Clase Main: Interfaz de Usuario por Consola 7

3. INTERFACES Y COMPORTAMIENTO DEL NEGOCIO 8

- 3.1. Contrato e Implementación de la Interfaz Calculable 8
- 3.2. Polimorfismo y Reglas para el Cálculo de Montos 9

4. MODELO DE DATOS Y PERSISTENCIA EN RED (MySQL) 10

- 4.1. Mapeo de la Clase Abstracta Base en la Base de Datos 10
- 4.2. Estrategia de Eliminación mediante Baja Lógica (Soft Delete) 11
- 4.3. Gestión Avanzada de Pedidos mediante Transacciones JDBC 12
 - 4.3.1. Control de Confirmación Manual (setAutoCommit) 12
 - 4.3.2. Mecanismo de Reatribución en Fallos (rollback) 13

5. CONFIGURACIÓN DEL ENTORNO MEDIANTE XML 14

- 5.1. Centralización de Parámetros en META-INF/persistence.xml 14
- 5.2. Ventajas del Acoplamiento Débil en las Credenciales de Acceso 15

6. CRITERIOS DE CALIDAD Y CONCLUSIONES

..... 16

- 6.1. Evaluación del Cumplimiento del Backlog de Historias de Usuario 16
- 6.2. Robustez del Sistema ante Excepciones y Errores de Entrada de Datos 17

• 7. Conclusión Final y Aprendizajes en Programación 2	18
--	----

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Contextualización del Proyecto (*Food Store*)

El proyecto Food Store nace de la necesidad de sistematizar los procesos de un negocio gastronómico en expansión. El software está diseñado como una aplicación de escritorio por consola que interactúa en tiempo real con una base de datos relacional. Permite centralizar el control de lo que vende el comercio, organizar quiénes compran y operan, y registrar el flujo exacto de los pedidos que ingresan a la cocina.

1.2. Objetivos Generales y Académicos

- Modularización: Dividir un sistema informático complejo en bloques lógicos independientes y con responsabilidades únicas.
- Persistencia Real: Abandonar el almacenamiento temporal en memoria (como las listas estáticas que se borran al cerrar el programa) e incorporar el almacenamiento permanente mediante bases de datos relacionales SQL.
- Abstracción y Diseño: Traducir un diagrama de clases del estándar UML a código Java funcional y tipado de forma estricta.

1.3. Alcance y Límites del Sistema de Consola

El sistema cubre las operaciones CRUD completas (Crear, Leer, Actualizar y Eliminar) para Categorías, Productos, Usuarios y Pedidos. Como límites explícitos determinados por la cátedra para Programación 2, no incluye una interfaz gráfica de usuario (GUI) ni tecnologías web (como API REST o HTML/CSS), enfocando todo el potencial en la lógica algorítmica y la correcta comunicación de datos a través de JDBC. Tampoco implementa un sistema de login con contraseña; el acceso a los diferentes roles se gestiona directamente desde opciones paramétricas del menú.

2. ARQUITECTURA DEL SISTEMA Y ESTRUCTURA DE CAPAS

2.1. Justificación del Diseño Orientado a Objetos (POO)

La arquitectura de este software se basa en el principio de Separación de Responsabilidades (SoC). Al separar el programa en capas, nos aseguramos de que los cambios en un sector (por ejemplo, si cambiamos de MySQL a otra base de datos) no afecten en absoluto a la interfaz de consola del usuario. Esto permite un código mantenible, escalable y fácil de testear.

2.2. Descripción de Paquetes y Responsabilidades

2.2.1. Paquete config: Inicialización y Conexión

Aloja la clase utilitaria ConexionDB. Utiliza técnicas de lectura de flujos de datos (InputStream) y parseo estructurado del estándar DOM para leer el archivo persistence.xml al iniciar la aplicación. Centraliza la apertura de conexiones JDBC para todo el proyecto.

2.2.2. Paquete entities: Modelo de Dominio y Herencia

Contiene las clases del negocio que mapean la realidad del negocio gastronómico. Es el núcleo de la POO en el proyecto; define la estructura y el comportamiento de los datos mediante encapsulamiento estricto (atributos privados con métodos accesorios públicos Getters y Setters).

2.2.3. Paquete enums: Enumeraciones Estructuradas

Contiene las clases de tipo enum (Rol, Estado, FormaPago). Su función es restringir los valores permitidos para ciertos atributos estratégicos del sistema, evitando que se inyecten datos inválidos en la base de datos (por ejemplo, asegurar que un pedido solo pueda estar en un estado predeterminado como PENDIENTE o ENTREGADO).

2.2.4. Paquete dao: Patrón de Acceso a Datos (JDBC)

El patrón DAO (Data Access Object) separa la lógica de acceso a datos de la lógica de negocios. Este paquete define interfaces claras y sus respectivas implementaciones concretas en las que se escribe código SQL puro (PreparedStatement y ResultSet). Es el único paquete que interactúa directamente con el motor MySQL.

2.2.5. Paquete service: Capa de Lógica y Validaciones

Esta capa actúa como un filtro de seguridad bi-direccional. Antes de que el DAO envíe información a guardar en la base de datos, el servicio valida las reglas del negocio (ej. verificar que un email tenga un formato válido o que el precio de un plato de comida sea mayor a cero). Si la validación falla, interrumpe el flujo y reporta el problema.

2.2.6. Paquete exception: Manejo de Errores Personalizados

Reemplaza el uso de excepciones genéricas de Java por excepciones propias del negocio informático (como EntityNotFoundException o ValidacionException). Esto permite capturar fallos específicos de forma controlada y mostrar mensajes comprensibles al usuario en lugar de romper el programa con trazas de error nativas.

2.2.7. Clase Main: Interfaz de Usuario por Consola

Representa la capa de vista del sistema. Maneja de manera recursiva bucles while y condicionales switch para pintar menús interactivos en la pantalla. Captura los datos ingresados por teclado mediante la clase java.util.Scanner y muestra los resultados formateados en cadenas de texto legibles.

3. INTERFACES Y COMPORTAMIENTO DEL NEGOCIO

3.1. Contrato e Implementación de la Interfaz Calculable

La interfaz Calculable define un contrato de comportamiento uniforme para cualquier entidad que requiera cálculos monetarios. Expone un único método abstracto:

```
Java  
double calcularTotal();
```

La clase Pedido implementa este método de forma obligatoria. Al activarse, la función inicializa un acumulador en cero y recorre mediante un bucle cada elemento de la colección interna de objetos DetallePedido, extrayendo sus subtotales y consolidando el precio definitivo.

3.2. Polimorfismo y Reglas para el Cálculo de Montos

El subtotal de cada línea de compra se calcula de forma automática al asociar un artículo. La regla matemática estricta es:

$\text{Subtotal} = \text{Cantidad} \times \text{Precio del Producto}$

Mediante el uso de polimorfismo, el sistema delega la responsabilidad de calcular los montos a los objetos correspondientes. El servicio de Pedidos simplemente invoca a calcularTotal() sin necesidad de conocer la lógica interna de cómo se estructuran las líneas del carrito de compras.

4. MODELO DE DATOS Y PERSISTENCIA EN BASE DE DATOS (MySQL)

4.1. Mapeo de la Clase Abstracta Base en la Base de Datos

La clase abstracta Base agrupa los atributos transversales de auditoría (id, eliminado, createdAt). En el modelo físico de MySQL, estos atributos se replicaron manualmente como columnas en cada una de las tablas del sistema. El atributo id se mapeó como clave primaria de tipo BIGINT con propiedades de incremento automático (AUTO_INCREMENT), lo que garantiza la unicidad de cada registro sin intervención del usuario.

4.2. Estrategia de Eliminación mediante Baja Lógica (*Soft Delete*)

Para proteger la integridad de los datos históricos del restaurante, se prohibió el borrado físico de filas (DELETE FROM). El sistema opera bajo la modalidad de Baja Lógica: cuando un operador decide eliminar una categoría, un usuario o un producto, el DAO ejecuta una consulta de actualización (UPDATE) sobre el registro modificando la bandera booleana eliminado = true. Para el usuario el dato deja de existir, pero a nivel de auditoría la información permanece intacta.

4.3. Gestión Avanzada de Pedidos mediante Transacciones JDBC

4.3.1. Control de Confirmación Manual (setAutoCommit)

La inserción de un pedido completo abarca múltiples modificaciones en paralelo (un registro en la tabla pedido y N registros en la tabla detalle_pedido). Para evitar fallos en los que se guarde el pedido pero se pierdan los productos de la compra, se desactivó el comportamiento por defecto de JDBC invocando `connection.setAutoCommit(false)`, lo que abre una transacción lógica unitaria.

4.3.2. Mecanismo de Reatribución en Fallos (rollback)

Si todas las operaciones se ejecutan satisfactoriamente, el bloque de código de la persistencia confirma los cambios de forma simultánea e irreversible mediante un `connection.commit()`. En contraposición, si el bucle encuentra un error crítico (como la falta de stock o la desconexión del servidor) el bloque catch atrapa el evento e invoca un `connection.rollback()`, restaurando la base de datos al estado exacto previo al inicio de la operación.

5. CONFIGURACIÓN DEL ENTORNO MEDIANTE XML

5.1. Centralización de Parámetros en META-INF/persistence.xml

El archivo `persistence.xml` actúa como la bitácora técnica de conectividad de la aplicación. En él se declaran de manera formal y estructurada en formato XML las cuatro propiedades obligatorias de la persistencia relacional: la dirección IP y puerto del motor de base de datos (url), las credenciales del usuario administrador (user y password), y la firma del conector de Java (driver).

5.2. Ventajas del Acoplamiento Débil en las Credenciales de Acceso

Al extraer los datos de acceso del código de Java y depositarlos en un archivo XML externo, el software adquiere una propiedad de diseño conocida como acoplamiento débil. Esto significa que si el administrador del restaurante decide cambiar la contraseña de la base de datos por motivos de seguridad, o si el software se traslada de una computadora de desarrollo a un servidor de producción, no es

necesario recompilar ni modificar una sola línea de código Java; basta con editar el archivo de texto XML.

6. CRITERIOS DE CALIDAD Y CONCLUSIONES

6.1. Evaluación del Cumplimiento del Backlog de Historias de Usuario

El sistema desarrollado cubre con éxito el mapa de historias de usuario (HUs) planteado en el diseño preliminar del proyecto gastronómico. El menú principal enlaza de forma directa las opciones operativas que permiten la administración limpia de las entidades secundarias (Categorías, Productos y Usuarios) y el procesamiento integral del flujo principal del negocio (Pedidos).

6.2. Robustez del Sistema ante Excepciones y Errores de Entrada de Datos

Para garantizar la continuidad operativa y evitar interrupciones abruptas durante el uso del menú de consola, la capa de presentación fue blindada mediante estructuras interceptoras. Las lecturas de teclado del objeto Scanner que esperan valores numéricos se encuentran encapsuladas en bloques de captura capaces de neutralizar eventos del tipo `InputMismatchException` (cuando el usuario tipea texto por error), limpiando el búfer y solicitando el reingreso seguro del dato.

7. Conclusión Final y Aprendizajes en Programación 2

El desarrollo del proyecto integrador Food Store representó una experiencia de aprendizaje clave en mi formación técnica. Permitted comprender la importancia del orden y evidenció cómo la programación orientada a objetos adquiere un valor real al complementarse con la persistencia relacional en bases de datos. El resultado final es una aplicación robusta, organizada bajo estándares universitarios y lista para adaptarse a futuros entornos de interfaces de usuario avanzadas.

VIDEO: <https://youtu.be/DgtB2YITRPc>